Student Result System

Cornerstone project work submitted to Aditya University in partial fulfilment of the requirements for the award of the Degree of

B.Tech. degree

in

Computer Science & Engineering (2026-2027)

SUBMITTED BY

S.L.V. KRISHNA REDDY	**25B11CS861**
P. SURYA AKHIL	**25B11CS717**
K.S.R. MOKSHAGNA	**25B11CS453**
K. VITESH KRISHNA	**25B11CS483**

UNDER THE GUIDANCE OF

M Sushma Devi

Assistant Professor

Department of CSE

Department of Computer Science & Engineering
Aditya University, Aditya Nagar, ADB Road,
Surampalem, Kakinada, District, Andhra Pradesh, India - 533437

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ADITYA UNIVERSITY, ADITYA NAGAR,
SURAMPALEM.



CERTIFICATE

This is to certify that the cornerstone project entitled "STUDENT RESULT SYSTEM" is being submitted by by SATTI LEELA VENKATA KRISHNA REDDY, PARUPALLI SURYA AKHIL, KONAGALLA SURYA RAJA MOKSHAGNA, KOTHAPALLI VITESH KRISHNA in partial fulfilment for the award of B. Tech Degree from Aditya University, is a record of original work done by them under my supervision. The contents of this project, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Project Guide
M Sushma Devi
Assistant Professor
Department of CSE
Aditya University

DECLARATION

We hereby declare that the Cornerstone project work entitled “STUDENT RESULT SYSTEM” is being submitted by us for the award of the B. Tech Degree in Computer Science and Engineering, Aditya University, Aditya Nagar, ADB Road, Surampalem is a record of original work carried out by us. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the Institutional Academic Integrity and AI Usage Policy.

Submitted By

S.L.V. KRISHNA REDDY 25B11CS861

P. SURYA AKHIL 25B11CS717

K.S.R. MOKSHAGNA 25B11CS453

K. VITESH KRISHNA 25B11CS483

Place: Surampalem

Date:

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **M Sushma Devi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K Chandra Sekhar, Associate Professor, Dr. Hari Jyothula, Associate Professor and Dr. P S Satya Prasad, Assistant Professor,** Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. A Phani Sridhar, Assistant Professor & HOD**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **DR. M.V. RAJESH, ASSOCIATE DEAN, SCHOOL OF COMPUTING**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **DR. G. SURESH, REGISTRAR; DR. A. RAMESH, PRO VICE-CHANCELLOR (ENGINEERING & SCIENCES); DR. S. RAMA SREE, PRO VICE-CHANCELLOR (ACADEMICS); DR. M.B. SRINIVAS, VICE-CHANCELLOR; AND DR. M. SREENIVASA REDDY, DEPUTY PRO CHANCELLOR AND MANAGEMENT, ADITYA UNIVERSITY**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **FACULTY MEMBERS, LAB TECHNICIANS, NON-TEACHING STAFF, AND OUR FRIENDS** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Name of the Students:

S.L.V.KRISHNA REDDY , P.SURYA AKHIL , K .S.R. MOKSHAGNA , K .VITESH KRISHNA

Date:

ABSTRACT

The Student Result System is a simple C++ based application designed to manage student academic information and generate results efficiently. The system stores student details using an array of objects and allows the user to enter important information such as roll number, student name, section, number of subjects, and marks obtained in each subject. To ensure reliable data entry, the system validates the entered information, including student names, sections, and marks, and prevents duplicate roll numbers from being entered.

After collecting the required details, the system calculates the total marks, percentage, and corresponding grade based on the marks obtained. The complete result of each student is then displayed in a clear and organized format. The system also supports different numbers of subjects for different students, making it flexible and suitable for basic academic result management.

This project demonstrates the practical implementation of important Object-Oriented Programming concepts in C++, including classes, objects, member functions, arrays, functions, data validation, and modularization. It provides a simple, efficient, and user-friendly method for maintaining student results during program execution. The system operates entirely in memory and does not use file storage, making it suitable for understanding fundamental C++ programming concepts.

ABBREVIATIONS

Abbreviation	Full Form / Use
C++	C Plus Plus
OOP	Object-Oriented Programming
I/O	Input/Output
#include	Preprocessor directive that adds a header file to the program before compilation.
<iostream>	Input/Output Stream – provides cin and cout for reading input from the keyboard and displaying output on the console.
<string>	String Library – provides the string class and its functions, such as length(), used to store and process roll numbers, names and sections.
<cctype>	C Character Type – provides character-handling functions such as toupper(), isalpha() and isalnum(), used for input validation and for converting roll numbers and sections to uppercase.
<iomanip>	Input/Output Manipulators – provides setprecision() to display the percentage with two decimal places.
<sstream>	String Stream – provides stringstream, used to check that the entered text is a valid whole number.
std	Standard namespace (using namespace std;) – gives direct access to cin, cout, string and other standard library names.

LIST OF FIGURES

Fig.No	Figure	Pages
1.4.1	Flowchart of Student Result System	3
2.3.1	Main Menu of Student Result System	13
2.3.2	Student Details and Marks Entry	13
2.3.3	Input Validation	14
2.3.4	Result Display	14

LIST OF TABLES

Table.No.	Table	Pages
1.3.1	Planned Modules for Implementation	1
2.1.1	OOP Concepts Used for Implementation	4
2.2.1	Student Class Member Functions	5
2.2.2	Input Validation Rules	12
2.2.3	Grade Classification	12

TABLE OF CONTENTS

S.No.	Contents	Pages
1	Title Page	i
2	Certificate	ii
3	Declaration	iii
4	Acknowledgement	iv
5	Abstract	v
6	Abbreviations	vi
7	List of Figures	vii
8	List of Tables	viii
Chapter 1	**PROJECT PLANNING**	**1–3**
1.1	Problem Statement	1
1.2	Project Objectives	1
1.3	Planned Modules	1
1.4	System Architecture (Algorithm & Flowchart)	2
1.4.1	Algorithm	2
1.4.2	Flowchart	3
Chapter 2	**PROJECT IMPLEMENTATION**	**4–14**
2.1	OOP Concepts Used for Implementation	4
2.2	Code and Functions	4
2.2.1	Student Class	4
2.2.2	Member Functions	5
2.2.3	Code	5
2.2.4	Input Validation	12
2.2.5	Grade Classification	12
2.3	Output Screens	12
2.3.1	Main Menu of Student Result System	13
2.3.2	Student Details and Marks Entry	13
2.3.3	Input Validation	14
2.3.4	Result Display	14
Chapter 3	**CONCLUSION & FUTURE SCOPE**	**15–16**
3.1	Conclusion	15
3.2	Future Scope	16

CHAPTER 1
PROJECT PLANNING

1.1 Problem Statement

Managing student academic results manually can lead to incorrect data entry, repeated calculations, duplicate student records and difficulty in maintaining consistent result formats. A simple application is required to store student details, accept subject-wise marks, validate the entered information and generate results accurately.

The Student Result System addresses this requirement through a menu-driven C++ program. The system provides separate operations for adding student details, adding marks, viewing results and exiting the application. Input validation is included to reduce invalid entries and provide clear error messages.

1.2 Project Objectives

- To develop a simple menu-driven Student Result System using C++.
- To apply Object-Oriented Programming concepts through a Student class and Student objects.
- To store student details and subject-wise marks in memory using an array of objects.
- To validate roll numbers, names, sections, subject counts, marks and menu choices.
- To prevent duplicate roll numbers from being registered.
- To calculate total marks, percentage, grade and PASS/FAIL status automatically.
- To display student results in a clear and structured format.

1.3 Planned Modules

Module	Use
Student Details Module	Collects and validates roll number, name, section and number of subjects.
Marks Management Module	Accepts, validates and stores marks for each subject.
Result Processing Module	Calculates total, percentage, grade and PASS/FAIL status.
Result Display Module	Displays complete student details, marks and final result.
Input Validation Module	Checks user input and displays appropriate error messages.
Student Search Module	Searches for a student using the normalized roll number.
Main Menu Module	Controls the overall program using four menu options.

Table 1.3.1 – Planned Modules for Implementation

1.4 System Architecture (Algorithm & Flowchart)

The system follows a simple sequential workflow controlled by the main menu. The user selects an operation, the corresponding function performs the required task, and control returns to the main menu until the Exit option is selected.

1.4.1 Algorithm

1. Start the program.
2. Display the Student Result System main menu.
3. Read and validate the user's menu choice.
4. If Add Student Details is selected, collect and validate student details and then enter marks.
5. If Add Marks is selected, validate the roll number, search the student array and enter marks for the selected student.
6. If View Results is selected, validate the roll number, search the student array and display the calculated result.
7. If Exit is selected, display the exit message and terminate the program.
8. Repeat the menu until the user selects Exit.

1.4.2 Flowchart

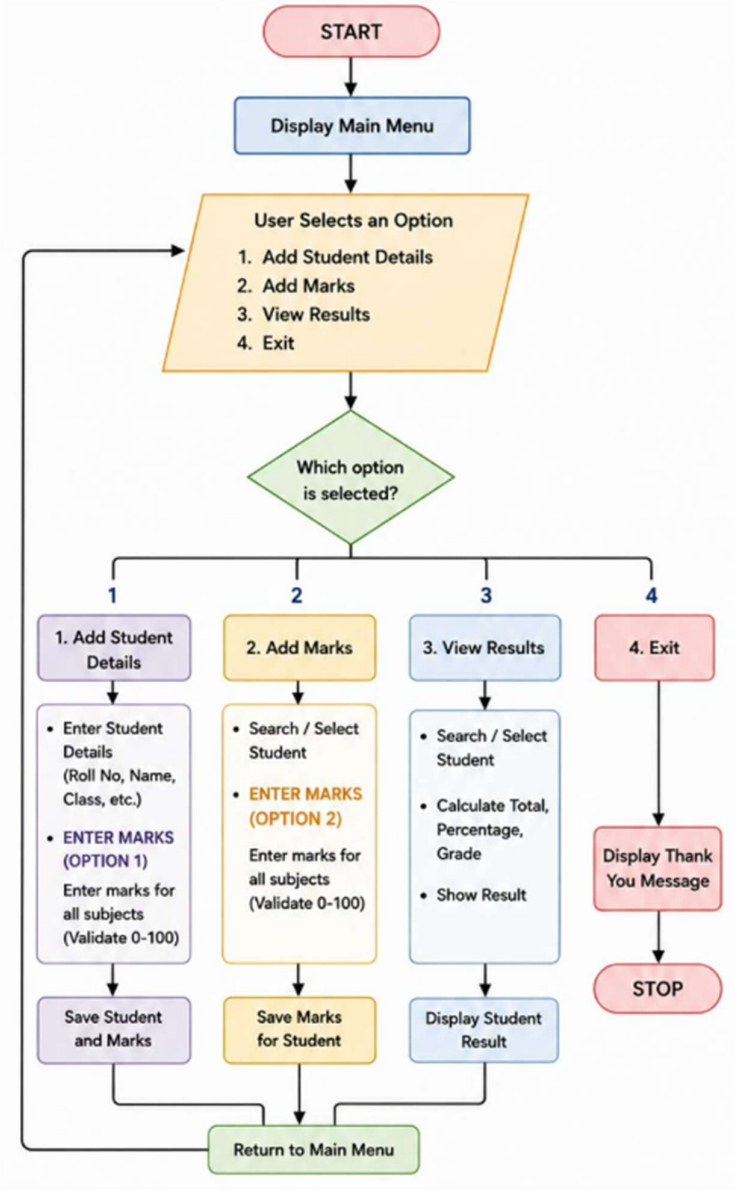


Fig. 1.4.1 – Flowchart of Student Result System

CHAPTER 2
PROJECT IMPLEMENTATION

2.1 OOP Concepts Used for Implementation

OOP Concept	Implementation in Project
Class	The Student class groups student data and related operations.
Objects	Student students[100] creates an array of Student objects.
Encapsulation	Student data members and related member functions are grouped inside the Student class.
Member Functions	addDetails(), addMarks() and displayResult() operate on Student objects.
Abstraction	Detailed validation and result-processing operations are handled through functions.
Scope Resolution Operator	Student::addDetails(), Student::addMarks() and Student::displayResult() define member functions outside the class.

Table 2.1.1 – OOP Concepts Used for Implementation

The project primarily demonstrates class, objects, encapsulation, member functions and abstraction. Inheritance and polymorphism are not used in the current implementation.

2.2 Code and Functions

2.2.1 Student Class

```
class Student
{
public:
   string roll;
   int subjects;
   int marks[10];
   string name;
   string section;

   void addDetails();
   void addMarks();
   void displayResult();
};
```

The Student class represents one student. It contains data members for roll number, number of subjects, marks, name and section. The three member functions perform student-detail entry, marks entry and result display.

2.2.2 Member Functions

Member Function	Use
addDetails()	Accepts and validates roll number, name, section and number of subjects, then calls addMarks().
addMarks()	Accepts subject-wise marks and validates that every mark is numeric and within 0–100.
displayResult()	Calculates total, percentage, grade and PASS/FAIL, then displays the formatted result.

Table 2.2.1 – Student Class Member Functions

2.2.3 Code

"STUDENT RESULT SYSTEM"

```
#include <iostream>
#include <string>
#include <cctype>
#include <iomanip>
#include <sstream>

using namespace std;

class Student
{
public:
   string roll;
   int subjects;
   int marks[10];
   string name;
   string section;

   void addDetails();
   void addMarks();
   void displayResult();
};

Student students[100];
int count = 0;

string normalizeRoll(string roll)
{
   for (int i = 0; i < (int)roll.length(); i++)
   {
      roll[i] = toupper((unsigned char)roll[i]);
   }

   return roll;
}
```

```
bool validRollNumber(string roll)
{
   if (roll.length() == 0)
      return false;

   if (roll.length() > 10)
      return false;

   for (int i = 0; i < (int)roll.length(); i++)
   {
      if (!isalnum((unsigned char)roll[i]))
         return false;
   }

   return true;
}

bool validName(string name)
{
   if (name.length() == 0)
      return false;

   bool hasAlphabet = false;

   for (int i = 0; i < (int)name.length(); i++)
   {
      if (isalpha((unsigned char)name[i]))
      {
         hasAlphabet = true;
      }
      else if (name[i] == ' ')
      {
         if (i == 0)
            return false;

         if (i == (int)name.length() - 1)
            return false;

         if (name[i + 1] == ' ')
            return false;
      }
      else
      {
         return false;
      }
   }

   return hasAlphabet;
}
```

```
bool validSection(string section)
{
   return section.length() == 1 &&
          isalpha((unsigned char)section[0]);
}

string normalizeSection(string section)
{
   for (int i = 0; i < (int)section.length(); i++)
   {
      section[i] = toupper((unsigned char)section[i]);
   }

   return section;
}

bool rollExists(string roll)
{
   roll = normalizeRoll(roll);

   for (int i = 0; i < count; i++)
   {
      if (students[i].roll == roll)
         return true;
   }

   return false;
}

bool getValidInteger(string input, int &number)
{
   if (input.length() == 0)
      return false;

   stringstream ss(input);

   char extra;

   if (!(ss >> number))
      return false;

   if (ss >> extra)
      return false;

   return true;
}

void Student::addDetails()
{
   string input;
```

```
while (true)
{
   cout << "\nEnter Roll Number (Maximum 10 characters): ";
   cin >> roll;

   if (!validRollNumber(roll))
   {
      cout << "\nInvalid roll number format!\n";
      cout << "Roll number must contain only letters and numbers.\n";
      cout << "Maximum length is 10 characters.\n";
      cout << "Special characters and spaces are not allowed.\n";
   }
   else
   {
      roll = normalizeRoll(roll);

      if (rollExists(roll))
      {
         cout << "\nRoll number already exists!\n";
         cout << "Please enter another roll number.\n";
      }
      else
      {
         break;
      }
   }
}

cin.ignore();

while (true)
{
   cout << "Enter Name: ";
   getline(cin, name);

   if (!validName(name))
   {
      cout << "\nInvalid name format!\n";
      cout << "Name must contain alphabets and single spaces only.\n";
      cout << "Leading, trailing and multiple spaces are not allowed.\n";
   }
   else
   {
      break;
   }
}

while (true)
{
```

```
        cout << "Enter Section (A-Z): ";
        cin >> section;

        if (!validSection(section))
        {
            cout << "\nInvalid section!\n";
            cout << "Enter one alphabet only (A-Z).\n";
        }
        else
        {
            section = normalizeSection(section);
            break;
        }
    }

    while (true)
    {
        cout << "Enter Number of Subjects (1-10): ";
        cin >> input;

        if (!getValidInteger(input, subjects))
        {
            cout << "\nInvalid number of subjects!\n";
            cout << "Please enter a number between 1 and 10.\n";
        }
        else if (subjects < 1 || subjects > 10)
        {
            cout << "\nInvalid number of subjects!\n";
            cout << "Please enter a number between 1 and 10.\n";
        }
        else
        {
            break;
        }
    }

    addMarks();
}

void Student::addMarks()
{
    string input;

    for (int i = 0; i < subjects; i++)
    {
        while (true)
        {
            cout << "Enter Marks for Subject "
                 << i + 1 << " (0-100): ";
```

```
            cin >> input;

            if (!getValidInteger(input, marks[i]))
            {
                cout << "\nInvalid marks!\n";
                cout << "Please enter a numeric value between 0 and 100.\n";
            }
            else if (marks[i] < 0 || marks[i] > 100)
            {
                cout << "\nInvalid marks!\n";
                cout << "Marks must be between 0 and 100.\n";
            }
            else
            {
                break;
            }
        }
    }

    cout << "\nMarks entered successfully!\n";
}

void Student::displayResult()
{
    int total = 0;
    bool pass = true;

    for (int i = 0; i < subjects; i++)
    {
        total += marks[i];

        if (marks[i] < 40)
            pass = false;
    }

    float percentage = (float)total / subjects;

    string grade;

    if (percentage >= 90)
        grade = "A+";
    else if (percentage >= 80)
        grade = "A";
    else if (percentage >= 70)
        grade = "B";
    else if (percentage >= 60)
        grade = "C";
    else if (percentage >= 50)
        grade = "D";
    else if (percentage >= 40)
```

```
        grade = "E";
    else
        grade = "F";

    if (percentage < 40)
        pass = false;

    cout << "\n\n================================================";
    cout << "\n              STUDENT RESULT";
    cout << "\n================================================";

    cout << "\nRoll Number        : " << roll;
    cout << "\nName               : " << name;
    cout << "\nSection            : " << section;
    cout << "\nNumber of Subjects : " << subjects;

    cout << "\n------------------------------------------------";

    cout << "\nSubject\t\tMarks";
    cout << "\n------------------------------------------------";

    for (int i = 0; i < subjects; i++)
    {
        cout << "\nSubject " << i + 1
             << "\t" << marks[i];
    }

    cout << "\n------------------------------------------------";

    cout << "\nTotal              : "
         << total << " / " << subjects * 100;

    cout << "\nPercentage         : "
         << fixed << setprecision(2)
         << percentage << "%";

    cout << "\nGrade              : " << grade;

    cout << "\nResult             : "
         << (pass ? "PASS" : "FAIL");

    cout << "\n================================================\n";
}

int main()
{
    string input;
    string roll;
    int choice;
    int found;
```

```
do
{
   // ---------------------------------------------------
   // MAIN MENU
   // ---------------------------------------------------

   cout << "\n\n================================================";
   cout << "\n            STUDENT RESULT SYSTEM";
   cout << "\n================================================";
   cout << "\n1. Add Student Details";
   cout << "\n2. Add Marks";
   cout << "\n3. View Results";
   cout << "\n4. Exit";
   cout << "\n================================================";

   while (true)
   {
      cout << "\nEnter your choice: ";
      cin >> input;

      if (!getValidInteger(input, choice))
      {
         cout << "\nInvalid choice!";
         cout << " Please enter a number from 1 to 4.\n";
      }
      else if (choice < 1 || choice > 4)
      {
         cout << "\nInvalid choice!";
         cout << " Please enter a number from 1 to 4.\n";
      }
      else
      {
         break;
      }
   }

   switch (choice)
   {
   case 1:

      if (count >= 100)
      {
         cout << "\nStudent storage is full!";
         cout << " Maximum 100 students can be added.\n";
         break;
      }

      students[count].addDetails();
```

```
    count++;

    cout << "\nStudent added successfully!";

    break;

case 2:

    while (true)
    {
        cout << "\nEnter Roll Number: ";
        cin >> roll;

        if (!validRollNumber(roll))
        {
            cout << "\nInvalid roll number format!\n";
            cout << "Roll number must contain only letters and numbers.\n";
            cout << "Maximum length is 10 characters.\n";
            cout << "Special characters and spaces are not allowed.\n";
            continue;
        }

        roll = normalizeRoll(roll);

        break;
    }

    found = 0;

    for (int i = 0; i < count; i++)
    {
        if (students[i].roll == roll)
        {
            students[i].addMarks();
            found = 1;
            break;
        }
    }

    if (!found)
        cout << "\nStudent not found!";

    break;
case 3:

    while (true)
    {
        cout << "\nEnter Roll Number: ";
        cin >> roll;
```

```
            if (!validRollNumber(roll))
            {
                cout << "\nInvalid roll number format!\n";
                cout << "Roll number must contain only letters and numbers.\n";
                cout << "Maximum length is 10 characters.\n";
                cout << "Special characters and spaces are not allowed.\n";
                continue;
            }

            roll = normalizeRoll(roll);

            break;
        }

        found = 0;

        for (int i = 0; i < count; i++)
        {
            if (students[i].roll == roll)
            {
                students[i].displayResult();
                found = 1;
                break;
            }
        }

        if (!found)
            cout << "\nStudent not found!";

        break;
    case 4:

        cout << "\nThank you for using Student Result System!";

        break;
    }
  } while (choice != 4);

  return 0;
}
```

2.2.4 Input Validation

Input	Validation Rule
Roll Number	1–10 characters; letters and numbers only; no spaces/special characters; duplicate roll numbers rejected.
Name	Alphabets and single spaces only; no leading, trailing or consecutive spaces.
Section	Exactly one alphabetic character; stored in uppercase.
Number of Subjects	Numeric value from 1 to 10.
Marks	Numeric value from 0 to 100.
Menu Choice	Numeric value from 1 to 4.

Table 2.2.2 – Input Validation Rules

2.2.5 Grade Classification

Percentage	Grade
90 and above	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
40–49.99	E
Below 40	F

Table 2.2.3 – Grade Classification

2.3 Output Screens

The following output sequence represents the expected demonstration flow of the implemented program. Actual console screenshots can be inserted in these locations after execution in Dev-C++.

2.3.1 Main Menu of Student Result System

```
================================================
              STUDENT RESULT SYSTEM
================================================
1. Add Student Details
2. Add Marks
3. View Results
4. Exit
================================================
Enter your choice:
```

Fig. 2.3.1 – Main Menu of Student Result System

2.3.2 Student Details and Marks Entry

```
================================================
              STUDENT RESULT SYSTEM
================================================
1. Add Student Details
2. Add Marks
3. View Results
4. Exit
================================================
Enter your choice: 1

Enter Roll Number (Maximum 10 characters): 25B11CS861
Enter Name: Krishna Reddy
Enter Section (A-Z): A
Enter Number of Subjects (1-10): 5
Enter Marks for Subject 1 (0-100): 98
Enter Marks for Subject 2 (0-100): 96
Enter Marks for Subject 3 (0-100): 99
Enter Marks for Subject 4 (0-100): 100
Enter Marks for Subject 5 (0-100): 97

Marks entered successfully!

Student added successfully!
```

Fig. 2.3.2 – Student Details and Marks Entry

2.3.3 Input Validation

```
Enter Roll Number (Maximum 10 characters): 25@

Invalid roll number format!
Roll number must contain only letters and numbers.
Maximum length is 10 characters.
Special characters and spaces are not allowed.

Enter Roll Number (Maximum 10 characters):

Enter your choice: 1

Enter Roll Number (Maximum 10 characters): 25B11CS861
Enter Name: Kri@

Invalid name format!
Name must contain alphabets and single spaces only.
Leading, trailing and multiple spaces are not allowed.
Enter Name: Krishna Reddy
```

Fig. 2.3.3 – Input Validation

2.3.4 Result Display

```
================================================
             STUDENT RESULT SYSTEM
================================================
1. Add Student Details
2. Add Marks
3. View Results
4. Exit
================================================
Enter your choice: 3

Enter Roll Number: 25B11CS861


================================================
              STUDENT RESULT
================================================
Roll Number        : 25B11CS861
Name               : Krishna Reddy
Section            : A
Number of Subjects : 5
------------------------------------------------
Subject     Marks
------------------------------------------------
Subject 1   98
Subject 2   99
Subject 3   95
Subject 4   97
Subject 5   100
------------------------------------------------
Total              : 489 / 500
Percentage         : 97.80%
Grade              : A+
Result             : PASS
================================================
```

Fig. 2.3.4 – Result Display

CHAPTER 3
CONCLUSION & FUTURE SCOPE

3.1 Conclusion

The **Student Result System** was successfully developed as a simple C++ application for managing student academic details and generating results. The system allows the user to enter student information such as roll number, name, section, number of subjects, and marks. Student records are maintained using an array of objects, which allows multiple student details to be stored during program execution without using file storage. The system also allows each student to have a different number of subjects, making it flexible for managing different student records.

The system includes **input validation** to improve the accuracy and reliability of the information entered by the user. It validates student names and sections, checks marks within the valid range, and prevents duplicate roll numbers. The system also provides appropriate error messages whenever invalid data is entered, allowing the user to correct the input. These validation features help reduce incorrect data and make the application easier and safer to use.

The result generation process calculates the total marks, percentage, grade, and pass/fail status based on the marks entered for each student. Separate functions are used for adding student details, entering marks, and displaying results, which provides function modularization and keeps the program organized. The project also demonstrates important **Object-Oriented Programming** concepts, including **classes, objects, member functions, and arrays of objects,** providing practical experience in applying C++ concepts to a real-world academic problem.

Overall, the Student Result System provides a simple and organized approach to student result management while satisfying the required project objectives. It reduces manual calculation, improves data entry through validation, and presents student results in a clear format. The project demonstrates how basic C++ and OOP concepts can be combined to develop a practical, easy-to-understand application for managing student academic results.

3.2 Future Scope

- Add file handling or a database to permanently store student records.
- Provide edit and delete operations for student records.
- Generate printable or downloadable result reports.
- Add authentication for administrators and authorized users.
- Provide subject names instead of only subject numbers.
- Add attendance and other academic performance information.
- Develop a graphical user interface for easier interaction.
- Add search and filtering options for multiple students.